

Comparación de Modelos y Optimización de Pipeline en el Dataset Titanic

Integrantes del Proyecto

- **Mario Steve Ocaña Avilés**

Descripción: Se encargó de una parte de la programación y soporte en investigación de modelos, parte de la documentación.

- **Jerylene Isabel Martínez Rodríguez**

Descripción: Se encargó del diseño general del sistema y la planificación del proyecto, parte de programación (librerías, algoritmos, etc), depuración de errores, parte de la documentación.

- **Jeremy Fabian Contreras Chancay**

Descripción: Realizó las pruebas, validación de resultados y depuración de errores (gráficas, código), parte de la documentación.

- **Genesis Nathalia Mayorga Reyes**

Descripción: Se encargó de una parte de la programación. Gestionó la documentación y la presentación final del proyecto.

```
# =====
# FASE 0: CONFIGURACIÓN GLOBAL CENTRALIZADA
# =====
import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings('ignore')

# — Semilla global —————
SEED = 42
np.random.seed(SEED)

# — Paleta de colores (MATLAB style) —————
MATLAB_COLORS = [
    '#0072BD', '#D95319', '#EDB120', '#7E2F8E',
    '#77AC30', '#4DBEEE', '#A2142F'
]

# — Config Plotly (Gráficos interactivos) —————
CONFIG_PLOTLY = {
    'displayModeBar': True,
    'displaylogo': False,
    'responsive': True,
    'scrollZoom': True,
}

config_interactivo = CONFIG_PLOTLY

# — Rutas de datos —————
PATH_TRAIN = 'train.csv'
PATH_TEST = 'test.csv'
URL_TRAIN = 'https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv'
URL_TEST = 'https://raw.githubusercontent.com/agconti/kaggle-titanic/master/data/test.csv'

print('=' * 65)
print(' CONFIGURACIÓN GLOBAL INICIALIZADA')
print('=' * 65)
```

```
=====
CONFIGURACIÓN GLOBAL INICIALIZADA
=====
```

```
# =====
# FASE 1.1: CARGA DE DATOS
# =====
import os
import plotly.graph_objects as go
import plotly.express as px
from plotly.subplots import make_subplots

print("=" * 70)
print(" FASE 1: CARGA DE DATOS Y ANÁLISIS EXPLORATORIO (EDA)")
print("=" * 70)

# Intento 1: Archivos locales (Kaggle / tu máquina)
try:
    if os.path.exists(PATH_TRAIN) and os.path.exists(PATH_TEST):
        train_raw = pd.read_csv(PATH_TRAIN)
```

```

test_raw = pd.read_csv(PATH_TEST)
print("✅ Datos cargados desde archivos locales.")
else:
    raise FileNotFoundError("Archivos locales no encontrados.")
except Exception as e:
    # Intento 2: URLs públicas de GitHub (repositorio)
    try:
        print(f"⚠️ Fallback a GitHub: {e}")
        train_raw = pd.read_csv(URL_TRAIN)
        test_raw = pd.read_csv(URL_TEST)
        print("✅ Datos cargados desde GitHub.")
    except Exception as e2:
        # Intento 3: Seaborn como último recurso
        print(f"⚠️ Fallback a Seaborn: {e2}")
        import seaborn as sns
        train_raw = sns.load_dataset('titanic')
        train_raw.columns = [col.capitalize() for col in train_raw.columns]
        train_raw.rename(columns={'Sibsp': 'SibSp', 'Parch': 'Parch'}, inplace=True)
        test_raw = train_raw.drop('Survived', axis=1).sample(frac=0.3, random_state=SEED).reset_index(drop=True)
        print("✅ Datos de respaldo cargados (Seaborn).")

print(f"\n📊 Dimensiones detectadas:")
print(f"    • Train: {train_raw.shape}")
print(f"    • Test:  {test_raw.shape}")
print(f"    • Test:  {test_raw.shape}")
print(f"    • Test:  {test_raw.shape}")
display(train_raw.head())

# =====
# FASE 1.2: TARJETAS RESUMEN DEL DATASET
# =====
fig_cards = make_subplots(
    rows=1, cols=4,
    specs=[[{"type": "indicator"}] * 4],
    horizontal_spacing=0.05
)

for i, (title, value) in enumerate([
    ("Train Rows",    train_raw.shape[0]),
    ("Test Rows",    test_raw.shape[0]),
    ("Features",     train_raw.shape[1] - 1),
    ("Survival Rate", train_raw['Survived'].mean() * 100)
], 1):
    fig_cards.add_trace(go.Indicator(
        mode="number",
        value=value,
        title={"text": title, "font": {"size": 14}},
        number={"font": {"size": 28},
            "valueformat": ",d" if isinstance(value, int) else ".1f"
        }, row=1, col=i)

fig_cards.update_layout(
    height=180,
    title_text="📊 Resumen Estructural del Dataset",
    title_font_size=16,
    margin=dict(t=60, b=20, l=20, r=20)
)
fig_cards.show(config=CONFIG_PLOTLY)

# =====
# FASE 1.3: ANÁLISIS DE VALORES FALTANTES
# =====
missing = train_raw.isnull().mean().sort_values(ascending=True) * 100
missing = missing[missing > 0]

# Colores según severidad
colors_bar = []
for v in missing:
    if v > 50:
        colors_bar.append('#A2142F') # Rojo: crítico
    elif v > 10:
        colors_bar.append('#D95319') # Naranja: moderado
    else:
        colors_bar.append('#0072BD') # Azul: leve

fig_missing = go.Figure()
fig_missing.add_trace(go.Bar(
    x=missing.values,
    y=missing.index,
    orientation='h',
    marker_color=colors_bar,
    marker_line_color='white',
    marker_line_width=1.5,

```

```

        text=[f"{v:.1f}%" for v in missing.values],
        textposition='outside',
        textfont=dict(size=11, color='#1A2B3C'),
        hovertemplate="<b>%{y}</b><br>Missing: %{x:.1f}%<extra></extra>"
    ))

fig_missing.update_layout(
    title_text="⚠ Porcentaje de Valores Faltantes",
    xaxis_title="% Missing",
    yaxis_title="Variable",
    height=300,
    showlegend=False,
    xaxis_range=[0, missing.max() * 1.3],
    margin=dict(t=60, b=40, l=120, r=60)
)
fig_missing.show(config=CONFIG_PLOTLY)

# =====
# FASE 1.4: DISTRIBUCIÓN DE LA VARIABLE OBJETIVO (SURVIVED)
# =====
counts = train_raw['Survived'].value_counts()

fig_donut = go.Figure(data=[go.Pie(
    labels=['Died', 'Survived'],
    values=counts.values,
    hole=0.45,
    marker=dict(
        colors=['#A2142F', '#0072BD'],
        line=dict(color='white', width=2.5)
    ),
    textinfo='percent+label+value',
    textfont=dict(size=13, color='white'),
    hovertemplate="<b>%{label}</b><br>Count: %{value}<br>Percentage: %{percent}<extra></extra>"
)])

fig_donut.update_layout(
    title_text="🎯 Target: Survived",
    height=400,
    showlegend=True,
    legend=dict(orientation="h", yanchor="bottom", y=-0.1, xanchor="center", x=0.5),
    margin=dict(t=60, b=60, l=40, r=40)
)
fig_donut.show(config=CONFIG_PLOTLY)

# =====
# FASE 1.5: TASA DE SUPERVIVENCIA POR SEXO
# =====
sex_surv = train_raw.groupby(['Sex', 'Survived']).size().reset_index(name='count')
sex_surv_pct = sex_surv.copy()
sex_surv_pct['pct'] = sex_surv_pct.groupby('Sex')['count'].transform(lambda x: x / x.sum() * 100)
sex_surv_pct['Survived_label'] = sex_surv_pct['Survived'].map({0: 'Died', 1: 'Survived'})

fig_sex = px.bar(
    sex_surv_pct,
    x='Sex', y='pct',
    color='Survived_label',
    color_discrete_map={'Died': '#A2142F', 'Survived': '#0072BD'},
    text_auto='.1f',
    labels={'Sex': 'Sexo', 'pct': '% Supervivencia', 'Survived_label': 'Survived'},
    barmode='group'
)

fig_sex.update_layout(
    title_text="👤 Tasa de Supervivencia por Sexo",
    yaxis_title="% Supervivencia",
    yaxis_range=[0, 110],
    height=400,
    legend_title_text='Survived',
    margin=dict(t=60, b=40, l=60, r=40)
)
fig_sex.update_traces(marker_line_color='white', marker_line_width=1.5)
fig_sex.show(config=CONFIG_PLOTLY)

# =====
# FASE 1.6: CLASE (PCLASS) VS SUPERVIVENCIA
# =====
pclass_surv = train_raw.groupby(['Pclass', 'Survived']).size().reset_index(name='count')
pclass_surv['Survived_label'] = pclass_surv['Survived'].map({0: 'Died', 1: 'Survived'})
pclass_surv['Pclass_label'] = pclass_surv['Pclass'].map({1: '1st Class', 2: '2nd Class', 3: '3rd Class'})

fig_pclass = px.bar(
    pclass_surv,

```

```

x='Pclass_label', y='count',
color='Survived_label',
color_discrete_map={'Died': '#A2142F', 'Survived': '#0072BD'},
text_auto=True,
labels={'Pclass_label': 'Pclass', 'count': 'Nº pasajeros', 'Survived_label': 'Survived'},
barmode='stack'
)

fig_pclass.update_layout(
    title_text="🚢 Clase vs Supervivencia",
    yaxis_title="Nº pasajeros",
    height=400,
    legend_title_text='Survived',
    margin=dict(t=60, b=40, l=60, r=40)
)
fig_pclass.show(config=CONFIG_PLOTLY)

# =====
# FASE 1.7: TÍTULO SOCIAL (EXTRAÍDO DEL NOMBRE) VS SUPERVIVENCIA
# =====
def extract_title(name):
    try:
        return name.split(',')[1].split('.')[0].strip()
    except:
        return 'Other'

# Crear columnas temporales SOLO para el EDA
train_raw['Title'] = train_raw['Name'].apply(extract_title)
title_counts = train_raw['Title'].value_counts()
top_titles = title_counts[title_counts >= 10].index
train_raw['Title_clean'] = train_raw['Title'].apply(
    lambda x: x if x in top_titles else 'Other'
)

# Gráfico
title_surv = train_raw.groupby(['Title_clean', 'Survived']).size().reset_index(name='count')
title_surv['pct'] = title_surv.groupby('Title_clean')['count'].transform(lambda x: x / x.sum() * 100)
title_surv['Survived_label'] = title_surv['Survived'].map({0: 'Died', 1: 'Survived'})

fig_title = px.bar(
    title_surv,
    y='Title_clean', x='pct',
    color='Survived_label',
    color_discrete_map={'Died': '#A2142F', 'Survived': '#0072BD'},
    text_auto='.0f',
    labels={'Title_clean': 'Título', 'pct': '% Supervivencia', 'Survived_label': 'Survived'},
    barmode='stack'
)

fig_title.update_layout(
    title_text="🚢 Título Social vs Supervivencia",
    xaxis_title="% Supervivencia",
    xaxis_range=[0, 105],
    height=400,
    margin=dict(t=60, b=40, l=100, r=40)
)
fig_title.show(config=CONFIG_PLOTLY)

train_raw.drop(columns=['Title', 'Title_clean'], inplace=True, errors='ignore')

print("\n🔴 Hallazgos clave del EDA:")
print(" • Cabin: 77% missing → la AUSENCIA es información (proxy de clase baja)")
print(" • Mujeres: ~74% sobrevivieron vs Hombres: ~19%")
print(" • 'Master' (niños) tiene altísima supervivencia")
print(" • 1st Class tiene la mayor tasa de supervivencia")
print("\n✅ Columnas temporales del EDA eliminadas. train_raw está limpio para Fase 2.")
print("=" * 70)

```

```
=====
FASE 1: CARGA DE DATOS Y ANÁLISIS EXPLORATORIO (EDA)
=====
⚠ Fallback a GitHub: Archivos locales no encontrados.
✅ Datos cargados desde GitHub.

📊 Dimensiones detectadas:
• Train: (891, 12)
• Test:  (418, 11)
=====
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

📊 Resumen Estructural del Dataset

Train Rows

891

Test Rows

418

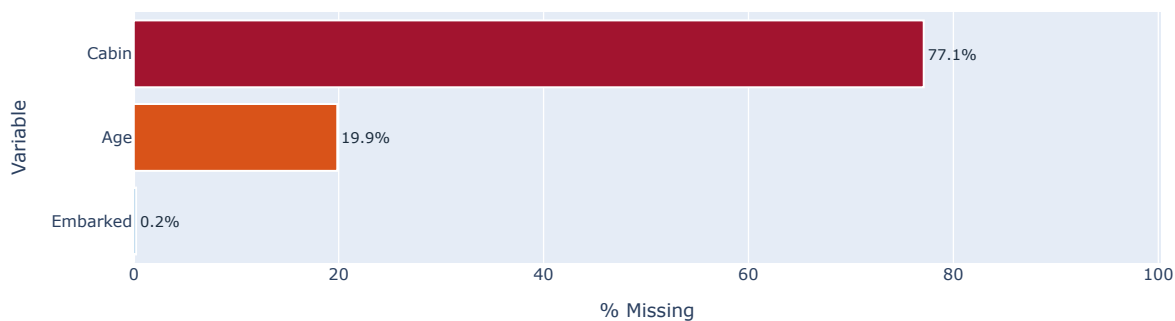
Features

11

Survival Rate

38.4

⚠ Porcentaje de Valores Faltantes



🎯 Target: Survived

```
# =====
# FASE 2: FEATURE ENGINEERING
# =====
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.utils.validation import check_is_fitted

class TitanicFeatureEngineer(BaseEstimator, TransformerMixin):
    """
    Transformer anti-leakage.
    Aprende estadísticas SOLO en fit(), las aplica en transform().
    """

    TITLE_MAPPING = {
        'Mr': 'Mr', 'Mrs': 'Mrs', 'Miss': 'Miss', 'Master': 'Master',
        'Mlle': 'Miss', 'Ms': 'Miss',
        'Countess': 'Rare', 'Lady': 'Rare', 'Sir': 'Rare',
        'Jonkheer': 'Rare', 'Dona': 'Rare', 'Don': 'Rare',
        'Major': 'Rare', 'Rev': 'Rare', 'Capt': 'Rare',
        'Col': 'Rare', 'Dr': 'Rare'
    }

    # Fare_Per_Person (evita multicolinealidad)
    COLS_TO_DROP = ['PassengerId', 'Name', 'Ticket', 'Cabin', 'Fare']

    FAMILY_BINS = [0, 1, 3, 20]
    FAMILY_LABELS = ['Solo', 'Small', 'Large']

    def __init__(self):
```

```

self.ticket_freq_ = None
self.fare_median_by_pclass_ = None

def fit(self, X, y=None):
    """Aprende estadísticas SOLO de X (train). Sin leakage."""
    self.ticket_freq_ = X['Ticket'].value_counts().to_dict()
    self.fare_median_by_pclass_ = X.groupby('Pclass')['Fare'].median().to_dict()
    return self

def transform(self, X, y=None):
    check_is_fitted(self, ['ticket_freq_', 'fare_median_by_pclass_'])
    X_ = X.copy()

    # 1. Título Social
    X_['Title'] = X_['Name'].str.extract(r'([A-Za-z]+)\.', expand=False)
    X_['Title'] = X_['Title'].map(self.TITLE_MAPPING).fillna('Rare')

    # 2. Estructura Familiar
    X_['Family_Size'] = X_['SibSp'] + X_['Parch'] + 1
    X_['Is_Alone'] = (X_['Family_Size'] == 1).astype(int)

    X_['Family_Category'] = pd.cut(
        X_['Family_Size'],
        bins=self.FAMILY_BINS,
        labels=self.FAMILY_LABELS,
        include_lowest=True
    ).astype(str) # convertir a str evita problemas con CategoricalDtype
                  # en el OneHotEncoder cuando hay categorías no vistas

    # 3. Tarifa por Persona (imputación sin leakage)
    if X_['Fare'].isnull().any():
        X_['Fare'] = X_.apply(
            lambda row: self.fare_median_by_pclass_.get(row['Pclass'], 0.0)
            if pd.isna(row['Fare']) else row['Fare'],
            axis=1
        )
    X_['Fare_Per_Person'] = X_['Fare'] / X_['Family_Size'].clip(lower=1)

    # 4. Cubierta del Cabin
    X_['Cabin_Deck'] = X_['Cabin'].apply(
        lambda x: str(x)[0]
        if pd.notna(x) and str(x).strip() != ''
        else 'Unknown'
    )
    # Validación defensiva: si la primera letra no es A-Z (dato corrupto),
    # se reemplaza por 'Unknown' en lugar de propagar un valor raro.
    valid_decks = set('ABCDEFGT')
    X_['Cabin_Deck'] = X_['Cabin_Deck'].apply(
        lambda d: d if d in valid_decks else 'Unknown'
    )

    # 5. Tamaño del Grupo de Ticket
    X_['Ticket_Group_Size'] = (
        X_['Ticket'].map(self.ticket_freq_).fillna(1).astype(int)
    )

    # 6. Eliminar columnas crudas
    X_.drop(
        columns=[c for c in self.COLS_TO_DROP if c in X_.columns],
        inplace=True
    )
    return X_

# Aplicar para visualización
X_viz = train_raw.drop('Survived', axis=1).copy()
y_viz = train_raw['Survived'].copy()

X_fe_viz = TitanicFeatureEngineer().fit_transform(X_viz)

# Reincorporar Survived solo para los gráficos de esta fase
train_fe = X_fe_viz.copy()
train_fe['Survived'] = y_viz.values

# --- 2.1 VISUALIZACIÓN ---
fig = make_subplots(
    rows=2, cols=2,
    subplot_titles=[
        'Distribución de Family_Size',
        'Supervivencia por Family_Category',
        'Distribución de Fare_Per_Person',
        'Supervivencia por Cabin_Deck'
    ]
),

```

```

        vertical_spacing=0.15,
        horizontal_spacing=0.10
    )

# Panel 1: Family_Size
fam_dist = train_fe['Family_Size'].value_counts().sort_index()
fig.add_trace(go.Bar(
    x=fam_dist.index, y=fam_dist.values, name='Family_Size',
    marker_color=MATLAB_COLORS[0],
    hovertemplate='Family Size: %{x}<br>Count: %{y}<extra></extra>'
), row=1, col=1)

# Panel 2: Family_Category
fam_cat_surv = train_fe.groupby(['Family_Category', 'Survived']).size().reset_index(name='count')
fam_cat_surv['pct'] = fam_cat_surv.groupby('Family_Category')['count'].transform(lambda x: x / x.sum() * 100)
fam_cat_surv = fam_cat_surv[fam_cat_surv['Survived'] == 1]
fig.add_trace(go.Bar(
    x=fam_cat_surv['Family_Category'], y=fam_cat_surv['pct'], name='Survival Rate',
    marker_color=MATLAB_COLORS[4],
    hovertemplate='Category: %{x}<br>Survival: %{y:.1f}%<extra></extra>'
), row=1, col=2)

# Panel 3: Fare_Per_Person
fig.add_trace(go.Histogram(
    x=train_fe['Fare_Per_Person'].clip(upper=100), nbinsx=40,
    marker_color=MATLAB_COLORS[2], opacity=0.7,
    hovertemplate='Fare/Person: %{x:.1f}<br>Count: %{y}<extra></extra>'
), row=2, col=1)

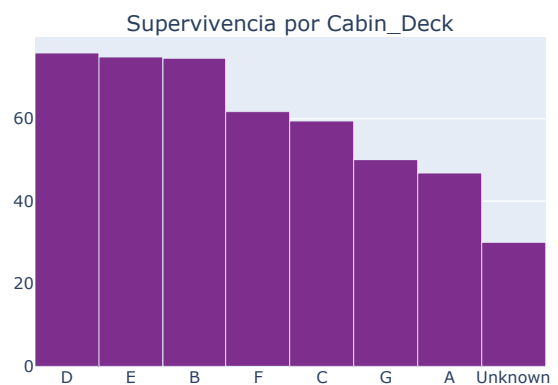
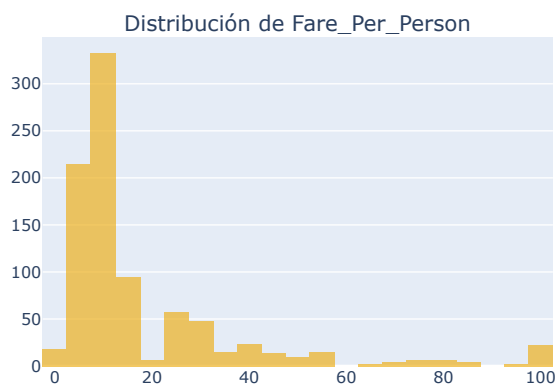
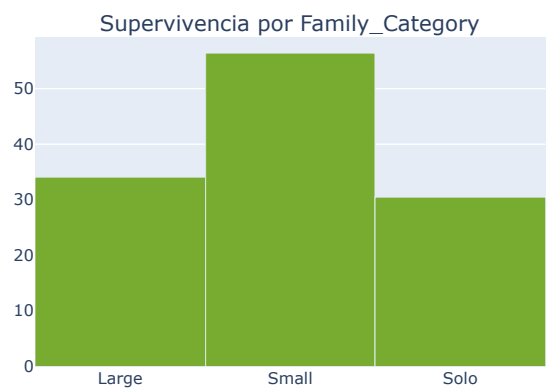
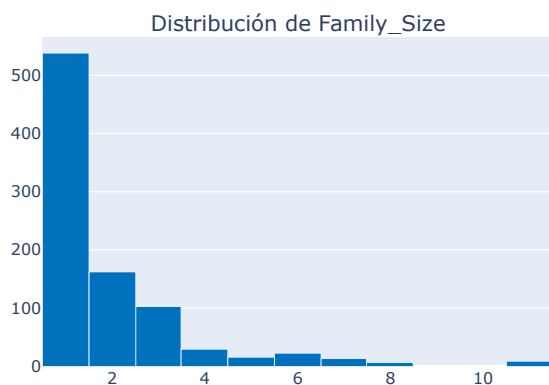
# Panel 4: Cabin_Deck
deck_surv = train_fe.groupby(['Cabin_Deck', 'Survived']).size().reset_index(name='count')
deck_surv['pct'] = deck_surv.groupby('Cabin_Deck')['count'].transform(lambda x: x / x.sum() * 100)
deck_surv = deck_surv[deck_surv['Survived'] == 1].sort_values('pct', ascending=False)
fig.add_trace(go.Bar(
    x=deck_surv['Cabin_Deck'], y=deck_surv['pct'], name='Deck Survival',
    marker_color=MATLAB_COLORS[3],
    hovertemplate='Deck: %{x}<br>Survival: %{y:.1f}%<extra></extra>'
), row=2, col=2)

fig.update_layout(
    height=700,
    title_text='🔗 Nuevas Features - Análisis de Distribución',
    title_font_size=16,
    showlegend=False,
    margin=dict(t=80, b=40, l=60, r=40)
)
fig.show(config=config_interactivo)

print("\n✅ Feature engineering completado. Nuevas variables creadas.")

```

Nuevas Features - Análisis de Distribución



✓ Feature engineering completado. Nuevas variables creadas.

```
# =====
# FASE 3: PIPELINE
# =====
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.base import clone

num_cols = [
    'Pclass', 'Age', 'SibSp', 'Parch',
    'Family_Size', 'Is_Alone',
    'Fare_Per_Person',          # <- Reemplaza a Fare
    'Ticket_Group_Size'
]
cat_cols = ['Sex', 'Embarked', 'Title', 'Family_Category', 'Cabin_Deck']

print('Columnas del Pipeline:')
print(f'  Numéricas ({len(num_cols)}): {num_cols}')
print(f'  Categóricas ({len(cat_cols)}): {cat_cols}')

# --- Transformadores numéricos y categóricos ---
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore',
                             sparse_output=False, drop='first'))
])

# Plantilla del preprocesador (NO se hace fit aquí)
preprocessor_template = ColumnTransformer(transformers=[
    ('num', numeric_transformer, num_cols),
    ('cat', categorical_transformer, cat_cols)
])

def build_pipeline(model):
```



```

"""
Construye un pipeline completo: FE + Preprocessing + Modelo.
"""
return Pipeline(steps=[
    ('feature_engineer', TitanicFeatureEngineer()),
    ('preprocessor',      clone(preprocessor_template)),
    ('classifier',        model)
])

# --- Split estratificado (único, prístino) ---
X = train_raw.drop('Survived', axis=1)
y = train_raw['Survived']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=SEED, stratify=y
)

# --- Estrategia de Validación Cruzada ---
cv_strategy = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)

print(f'\nSplit: {X_train.shape[0]} train | {X_test.shape[0]} test')
print(f'CV: {cv_strategy.n_splits}-Fold Estratificado')
print('✅ Pipeline construido. Cero Data Leakage garantizado.')

# =====
# FASE 3.2: VISUALIZACIÓN DEL EFECTO DEL SCALING
# =====
# Para ilustrar el efecto del StandardScaler SIN hacer leakage
# en el test set prístino, hacemos un split temporal aislado.

X_temp = train_raw[['Age']].copy()
X_tv, _, _ = train_test_split(
    X_temp, y, test_size=0.2, random_state=SEED, stratify=y
)

imp_viz = SimpleImputer(strategy='median')
scl_viz = StandardScaler()

age_original = imp_viz.fit_transform(X_tv[['Age']]).flatten()
age_procesada = scl_viz.fit_transform(age_original.reshape(-1, 1)).flatten()

fig_scaled = make_subplots(
    rows=2, cols=1, shared_xaxes=False,
    subplot_titles=("Distribución Original de 'Age'",
                    "Distribución Estandarizada (Z-score)"),
    vertical_spacing=0.15
)

fig_scaled.add_trace(go.Histogram(
    x=age_original, nbinsx=35,
    marker_color=MATLAB_COLORS[0], opacity=0.75,
    histnorm='probability density', name='Original'
), row=1, col=1)

fig_scaled.add_trace(go.Histogram(
    x=age_procesada, nbinsx=35,
    marker_color=MATLAB_COLORS[3], opacity=0.75,
    histnorm='probability density', name='Estandarizada'
), row=2, col=1)

fig_scaled.add_vline(
    x=float(np.median(age_original)), line_dash='dash',
    line_color=MATLAB_COLORS[1],
    annotation_text=f'Mediana={np.median(age_original):.0f}',
    row=1, col=1
)

fig_scaled.add_vline(
    x=0, line_dash='dash', line_color=MATLAB_COLORS[4],
    annotation_text='Media=0', row=2, col=1
)

fig_scaled.update_layout(
    title_text='Efecto del StandardScaler (Calculado solo sobre Train Viz)',
    height=600, showlegend=False, margin=dict(t=80, b=40, l=60, r=40)
)

fig_scaled.update_xaxes(title_text='Edad (años)', row=1, col=1)
fig_scaled.update_xaxes(title_text='Z-score', row=2, col=1)
fig_scaled.update_yaxes(title_text='Densidad', row=1, col=1)
fig_scaled.update_yaxes(title_text='Densidad', row=2, col=1)

fig_scaled.show(config=CONFIG_PLOTLY)

```

```

# =====
# FASE 4 & 5: MODELOS Y VALIDACIÓN ESTRATIFICADA
# =====
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from xgboost import XGBClassifier
from sklearn.model_selection import cross_validate

print("=" * 70)
print(" FASE 4 & 5: MODELOS Y VALIDACIÓN CRUZADA (5-FOLD)")
print("=" * 70)

# scale_pos_weight calculado sobre y_train (no y completo)
scale_pos_weight = (y_train == 0).sum() / (y_train == 1).sum()
print(f'📊 scale_pos_weight (ratio de desbalance): {scale_pos_weight:.3f}')

# — Diccionario de Modelos Base —————
models = {
    'Logistic Regression': LogisticRegression(
        max_iter=1000, class_weight='balanced', random_state=SEED
    ),
    'Decision Tree': DecisionTreeClassifier(
        class_weight='balanced', max_depth=6, min_samples_leaf=5, random_state=SEED
    ),
    'Random Forest': RandomForestClassifier(
        n_estimators=200, class_weight='balanced', max_depth=10,
        min_samples_leaf=2, random_state=SEED
    ),
    'Gradient Boosting': GradientBoostingClassifier(
        n_estimators=200, max_depth=4, learning_rate=0.1,
        subsample=0.8, random_state=SEED
    ),
    'XGBoost': XGBClassifier(
        n_estimators=200, max_depth=5, learning_rate=0.1,
        scale_pos_weight=scale_pos_weight,
        eval_metric='logloss', random_state=SEED
    ),
}

# Construir pipelines completos (FE + Preprocessing + Modelo)
pipelines = {name: build_pipeline(model) for name, model in models.items()}

```

```

# — Métricas de scoring —————
# Se incluye average_precision (PR-AUC) además de ROC-AUC
# porque con desbalance 62/38 el ROC-AUC puede ser optimista.
scoring_metrics = {
    'Accuracy': 'accuracy',
    'Precision_Macro': 'precision_macro',
    'Recall_Macro': 'recall_macro',
    'F1_Macro': 'f1_macro',
    'ROC_AUC': 'roc_auc',
    'PR_AUC': 'average_precision',
}

# — Ejecución de Validación Cruzada —————
cv_results_list = []

for name, pipe in pipelines.items():
    print(f' Validando {name}...')
    cv_res = cross_validate(
        pipe, X_train, y_train,
        cv=cv_strategy,
        scoring=scoring_metrics,
        n_jobs=-1
    )
    row = {'Modelo': name}
    for metric_name in scoring_metrics:
        scores = cv_res[f'test_{metric_name}']
        row[metric_name] = float(scores.mean())
        row[f'{metric_name}_std'] = float(scores.std())
    cv_results_list.append(row)

# Crear DataFrame y ordenar por ROC_AUC
df_cv = (
    pd.DataFrame(cv_results_list)
    .sort_values('ROC_AUC', ascending=False)
    .reset_index(drop=True)
)

print("\n✅ Validación cruzada completada.")
display(df_cv[['Modelo', 'Accuracy', 'F1_Macro', 'ROC_AUC', 'PR_AUC']].round(4))

# =====
# FASE 5.1: VISUALIZACIÓN DE RESULTADOS CV
# =====

# --- 5.1.1 TABLA COMPARATIVA CV ---
metricas_mostrar = [
    'Modelo', 'Accuracy', 'Precision_Macro',
    'Recall_Macro', 'F1_Macro', 'ROC_AUC', 'PR_AUC'
]
df_display = df_cv[metricas_mostrar].copy()

for col in df_display.columns[1:]:
    df_display[col] = df_display[col].apply(lambda x: f"{x:.4f}")

# Colores dinámicos: oro para el 1º, plata para el 2º
cell_colors = []
for col in df_display.columns:
    col_colors = []
    for idx in range(len(df_display)):
        if col == 'Modelo':
            if idx == 0: col_colors.append('#FFD700')
            elif idx == 1: col_colors.append('#C0C0C0')
            else: col_colors.append('#FFFFFF')
        else:
            col_colors.append('#FFFFFF')
    cell_colors.append(col_colors)

table_height = 35 * len(df_display) + 100

fig_table = go.Figure(data=[go.Table(
    header=dict(
        values=[f"<b>{c}</b>" for c in df_display.columns],
        fill_color=MATLAB_COLORS[0], align='center',
        font=dict(color='white', size=12), height=35
    ),
    cells=dict(
        values=[df_display[col] for col in df_display.columns],
        fill_color=cell_colors, align='center',
        font=dict(size=11, color='#2C3E50'), height=30
    )
)])

```

```

fig_table.update_layout(
    title_text='📊 Resultados Cross-Validation (5-Fold Estratificado) – incluye PR-AUC',
    height=table_height,
    margin=dict(t=60, b=20, l=20, r=20)
)
fig_table.show(config=CONFIG_PLOTLY)

# --- 5.1.2 GRÁFICO DE BARRAS AGRUPADAS (CON BARRAS DE ERROR) ---
fig_bars = go.Figure()

metricas_plot = ['Accuracy', 'Precision_Macro', 'Recall_Macro',
                  'F1_Macro', 'ROC_AUC', 'PR_AUC']
nombres_metricas = ['Accuracy', 'Precision', 'Recall', 'F1', 'ROC AUC', 'PR AUC']

for i, (metric, nombre) in enumerate(zip(metricas_plot, nombres_metricas)):
    fig_bars.add_trace(go.Bar(
        x=df_cv['Modelo'],
        y=df_cv[metric],
        name=nombre,
        marker_color=MATLAB_COLORS[i % len(MATLAB_COLORS)],
        text=[f"{v:.3f}" for v in df_cv[metric]],
        textposition='outside',
        textfont=dict(size=9, color='#2C3E50'),
        error_y=dict(
            type='data',
            array=df_cv[f'{metric}_std'],
            visible=True,
            thickness=1.5
        )
    ))

fig_bars.update_layout(
    barmode='group',
    title_text='📊 Comparación de Métricas por Modelo (CV=5)',
    xaxis_title='Modelo',
    yaxis_title='Score',
    yaxis_range=[0, 1.15],
    height=520,
    legend=dict(orientation="h", yanchor="bottom", y=1.02,
                xanchor="center", x=0.5),
    margin=dict(t=110, b=60, l=60, r=40)
)
fig_bars.show(config=CONFIG_PLOTLY)

# — Resultados finales —————
mejor_roc = df_cv.iloc[0]
mejor_pr = df_cv.loc[df_cv['PR_AUC'].idxmax()]

print(f"\n🏆 Mejor modelo por CV ROC-AUC : {mejor_roc['Modelo']} "
      f"({mejor_roc['ROC_AUC']:.4f} ± {mejor_roc['ROC_AUC_std']:.4f})")
print(f"🏆 Mejor modelo por CV PR-AUC : {mejor_pr['Modelo']} "
      f"({mejor_pr['PR_AUC']:.4f} ± {mejor_pr['PR_AUC_std']:.4f})")

""" Advertencia explícita sobre IC solapados ———
Las std de CV son ~0.02-0.04 para todos los modelos,
lo que implica IC que se solapan entre sí.
No existe un ganador estadísticamente significativo en CV.
La selección final se hará sobre el test set con bootstrap CI. """

print("\n⚠️ NOTA ESTADÍSTICA:")
print(" Las desviaciones estándar de CV se solapan entre modelos.")
print(" El ranking por ROC-AUC medio orienta la búsqueda de hiperparámetros,")
print(" pero NO determina un ganador hasta la evaluación en test set (Fase 7).")
print(" En Fase 7 se aplicará bootstrap CI + análisis de solapamiento de IC.")
print("=" * 70)

```

```
# =====
# FASE 6: OPTIMIZACIÓN DE HIPERPARÁMETROS
# =====
from sklearn.model_selection import RandomizedSearchCV, GridSearchCV

print("=" * 70)
print(" FASE 6: OPTIMIZACIÓN DE HIPERPARÁMETROS (CV=5)")
print("=" * 70)

top_3_models = df_cv.head(3)['Modelo'].tolist()
print(f' 🍷 Top 3 modelos por CV ROC-AUC: {top_3_models}')

print(f'\n 🌟 Justificación de scoring="roc_auc":')
print(f' • El desbalance (62/38) hace que Accuracy sea engañosa.')
print(f' • ROC-AUC evalúa la capacidad de ranking del modelo.')
print(f' • Es insensible al umbral de decisión (crítico en desbalance).')

# — Espacios de búsqueda (Param Grids) —————
param_grids = {
    'Random Forest': {
        'classifier__n_estimators': [100, 200, 300],
        'classifier__max_depth': [6, 8, 10, 12],
        'classifier__min_samples_split': [2, 5, 10],
        'classifier__min_samples_leaf': [1, 2, 4],
    },
    'XGBoost': {
        'classifier__n_estimators': [100, 200, 300],
        'classifier__max_depth': [3, 4, 5],
    },
}
```

```

        'classifier__learning_rate': [0.01, 0.05, 0.1, 0.2],
        'classifier__subsample': [0.7, 0.8, 0.9],
        'classifier__colsample_bytree': [0.7, 0.8, 0.9],
    },
    'Gradient Boosting': {
        'classifier__n_estimators': [100, 200, 300],
        'classifier__max_depth': [3, 4, 5],
        'classifier__learning_rate': [0.01, 0.05, 0.1],
        'classifier__subsample': [0.6, 0.7, 0.8],
    },
    'Logistic Regression': {
        'classifier__C': [0.001, 0.01, 0.1, 1, 10, 100],
        'classifier__penalty': ['l1', 'l2'],
        'classifier__solver': ['liblinear', 'saga'],
    },
    'Decision Tree': {
        'classifier__max_depth': [3, 4, 5, 6],
        'classifier__min_samples_split': [5, 10, 20],
        'classifier__min_samples_leaf': [2, 4, 8],
        'classifier__criterion': ['gini', 'entropy'],
    },
}

GAP_ALERTA = 0.05
GAP_CRITICO = 0.15

best_pipelines = {}
search_results = {}
gap_log = {}

# — Ejecución de la Búsqueda —————
for model_name in top_3_models:

    if model_name not in param_grids:
        print(f'\n⚠ Sin espacio de búsqueda definido para {model_name}, saltando.')
        continue

    # GB tiene 3x3x3=81 combinaciones
    # → GridSearchCV exhaustivo. RF y XGBoost tienen >200
    # combinaciones → RandomizedSearchCV correcto.
    if model_name == 'Gradient Boosting':
        n_combinaciones = 1
        for v in param_grids[model_name].values():
            n_combinaciones *= len(v)
        print(f'\n🔍 Optimizando {model_name} '
              f'f'(GridSearchCV exhaustivo, {n_combinaciones} combinaciones, cv=5)...')
        search = GridSearchCV(
            estimator=pipelines[model_name],
            param_grid=param_grids[model_name],
            scoring='roc_auc',
            cv=cv_strategy,
            n_jobs=-1,
            verbose=0,
            return_train_score=True
        )
    else:
        print(f'\n🔍 Optimizando {model_name} (RandomizedSearchCV, n_iter=50, cv=5)...')
        search = RandomizedSearchCV(
            estimator=pipelines[model_name],
            param_distributions=param_grids[model_name],
            n_iter=50,
            scoring='roc_auc',
            cv=cv_strategy,
            random_state=SEED,
            n_jobs=-1,
            verbose=0,
            return_train_score=True
        )

    search.fit(X_train, y_train)

    cv_mean = search.best_score_
    train_mean = search.cv_results_['mean_train_score'][search.best_index_]
    gap = train_mean - cv_mean
    gap_log[model_name] = gap

    print(f'✅ Best CV ROC-AUC : {cv_mean:.4f}')
    print(f'✅ Best Train ROC-AUC: {train_mean:.4f} (GAP: {gap:.4f})')
    print(f'✅ Best params: {search.best_params_}')

# — Acción correctiva según nivel de GAP —————
if gap > GAP_CRITICO:

```

```

print(f' 🚫 GAP CRÍTICO (>{GAP_CRITICO:.0%}). '
      f'Re-lanzando búsqueda con espacio más restrictivo...')

param_grids_restrictivos = {
    'Random Forest': {
        'classifier__n_estimators': [100, 200],
        'classifier__max_depth': [4, 6],
        'classifier__min_samples_split': [10, 20],
        'classifier__min_samples_leaf': [4, 8],
    },
    'XGBoost': {
        'classifier__n_estimators': [100, 200],
        'classifier__max_depth': [3, 4],
        'classifier__learning_rate': [0.01, 0.05],
        'classifier__subsample': [0.6, 0.7],
        'classifier__colsample_bytree': [0.6, 0.7],
    },
    'Gradient Boosting': {
        'classifier__n_estimators': [100, 200],
        'classifier__max_depth': [2, 3],
        'classifier__learning_rate': [0.01, 0.05],
        'classifier__subsample': [0.5, 0.6],
    },
    'Decision Tree': {
        'classifier__max_depth': [3, 4],
        'classifier__min_samples_split': [20, 30],
        'classifier__min_samples_leaf': [8, 10],
        'classifier__criterion': ['gini', 'entropy'],
    },
}

if model_name in param_grids_restrictivos:
    # GB restrictivo: combinaciones → Grid también
    if model_name == 'Gradient Boosting':
        search2 = GridSearchCV(
            estimator=pipelines[model_name],
            param_grid=param_grids_restrictivos[model_name],
            scoring='roc_auc',
            cv=cv_strategy,
            n_jobs=-1,
            verbose=0,
            return_train_score=True
        )
    else:
        search2 = RandomizedSearchCV(
            estimator=pipelines[model_name],
            param_distributions=param_grids_restrictivos[model_name],
            n_iter=30,
            scoring='roc_auc',
            cv=cv_strategy,
            random_state=SEED,
            n_jobs=-1,
            verbose=0,
            return_train_score=True
        )

search2.fit(X_train, y_train)

cv_mean2 = search2.best_score_
train_mean2 = search2.cv_results_['mean_train_score'][search2.best_index_]
gap2 = train_mean2 - cv_mean2
gap_log[model_name] = gap2

print(f' 🔄 Re-búsqueda completada.')
print(f' CV ROC-AUC : {cv_mean2:.4f} (antes: {cv_mean:.4f})')
print(f' Train ROC-AUC: {train_mean2:.4f} (GAP: {gap2:.4f})')

if gap2 < gap:
    print(f' ✅ Se adopta versión re-buscada (GAP reducido).')
    search = search2
    cv_mean = cv_mean2
else:
    print(f' ⚠️ Re-búsqueda no redujo el GAP. '
          f'Se mantiene versión original.')

elif gap > GAP_ALERTA:
    print(f' ⚠️ GAP > {GAP_ALERTA:.0%}. Overfitting leve. '
          f'Acceptable para dataset pequeño (n={len(X_train)}).')
    print(f' Considerar más datos o mayor regularización en producción.')

else:
    print(f' ✅ GAP dentro del límite aceptable (< {GAP_ALERTA:.0%}).')

```

```

best_pipelines[model_name] = search.best_estimator_
search_results[model_name] = search

print(f'\n✅ {len(best_pipelines)} modelos optimizados y guardados en "best_pipelines".')

# — Visualización —————
fig_opt = make_subplots(
    rows=2, cols=2,
    subplot_titles=[
        '📊 ROC-AUC: Base vs Optimizado',
        f'📈 Sensibilidad: {top_3_models[0]}',
        f'📈 Sensibilidad: {top_3_models[1]}',
        f'📈 Sensibilidad: {top_3_models[2]}'
        if len(top_3_models) > 2 else '📈 Sensibilidad: Modelo 3'
    ],
    vertical_spacing=0.18,
    horizontal_spacing=0.12,
    specs=[["type": "bar"], {"type": "bar"}],
    [{"type": "bar"}, {"type": "bar"}]
)

# — Panel 1: Base vs Optimizado con GAP anotado —————
modelos_opt = list(best_pipelines.keys())
auc_base = [df_cv[df_cv['Modelo'] == m]['ROC_AUC'].values[0] for m in modelos_opt]
auc_opt = [search_results[m].best_score_ for m in modelos_opt]
mejoras = [o - b for o, b in zip(auc_opt, auc_base)]

max_auc = max(max(auc_base), max(auc_opt))

fig_opt.update_yaxes(
    range=[0, max_auc + 0.12],
    row=1,
    col=1
)

fig_opt.add_trace(go.Bar(
    x=modelos_opt, y=auc_base, name='Base (CV)',
    marker_color=MATLAB_COLORS[0],
    text=[f'{v:.4f}' for v in auc_base],
    textposition='outside'
), row=1, col=1)

fig_opt.add_trace(go.Bar(
    x=modelos_opt, y=auc_opt, name='Optimizado (CV)',
    marker_color=MATLAB_COLORS[4],
    text=[f'{v:.4f}' for v in auc_opt],
    textposition='outside'
), row=1, col=1)

"""
for i, (m, mejora) in enumerate(zip(modelos_opt, mejoras)):
    color_txt = '#0072BD' if mejora > 0 else '#A2142F'
    fig_opt.add_annotation(
        text=f'Δ {mejora:+.4f} | GAP={gap_log.get(m, 0):.3f}',
        x=m, y=max(auc_base[i], auc_opt[i]) + 0.025,
        showarrow=False,
        font=dict(color=color_txt, size=10),
        row=1, col=1
    )
"""

# — Paneles 2-4: Sensibilidad de hiperparámetros —————
positions = [(1, 2), (2, 1), (2, 2)]

for idx, model_name in enumerate(top_3_models):
    if idx >= 3:
        break
    r, c = positions[idx]
    results_df = pd.DataFrame(search_results[model_name].cv_results_)
    param_impacts = {}

    for param_name in param_grids[model_name]:
        col_name = f'param_{param_name}'
        if col_name in results_df.columns:
            param_auc = (
                results_df.groupby(col_name)['mean_test_score']
                .mean()
                .sort_values(ascending=False)
            )
            param_impacts[param_name.replace('classifier__', '')] = param_auc

```



```

if param_impacts:
    best_param_name = max(
        param_impacts.keys(),
        key=lambda k: param_impacts[k].max() - param_impacts[k].min()
    )
    param_auc = param_impacts[best_param_name]
    x_vals = [str(v) for v in param_auc.index]
    y_vals = param_auc.values

    fig_opt.add_trace(go.Bar(
        x=x_vals, y=y_vals,
        marker_color=MATLAB_COLORS[idx % len(MATLAB_COLORS)],
        text=[f'{v:.4f}' for v in y_vals],
        textposition='outside',
        textfont=dict(size=9),
        name=f'{model_name} - {best_param_name}',
        showlegend=False
    ), row=r, col=c)

    fig_opt.update_xaxes(title_text=best_param_name, row=r, col=c)
    fig_opt.update_yaxes(title_text='ROC-AUC medio', row=r, col=c)

fig_opt.update_layout(
    height=850,
    title_text='🌀 Impacto de la Optimización de Hiperparámetros',
    title_font_size=16,
    barmode='group',
    legend=dict(orientation='h', yanchor='bottom', y=1.08,
                xanchor='center', x=0.5),
    margin=dict(t=140, b=60, l=60, r=40)
)
fig_opt.show(config=CONFIG_PLOTLY)

print(f'\n✅ {len(best_pipelines)} modelos optimizados y guardados en "best_pipelines".')
print("=" * 70)
print("\n🔥 INTERPRETACIÓN DE LA OPTIMIZACIÓN:")
print("    • Panel 1: mejora por tuning + GAP final de cada modelo.")
print("    • GAP > 15%: re-búsqueda automática con espacio restrictivo.")
print("    • GAP 5-15%: aceptable en datasets pequeños (n < 1000).")
print("    • GAP < 5%: modelo bien generalizado.")
print("    • Paneles 2-4: hiperparámetro MÁS influyente de cada modelo.")
print("=" * 70)

```

```
# =====
# FASE 7.1: CONSTRUCCIÓN DEL CONJUNTO FINAL DE PIPELINES
# =====
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, average_precision_score
)
from sklearn.base import clone

print("=" * 70)
print(" FASE 7: EVALUACIÓN FINAL EN TEST SET (COMPARACIÓN JUSTA)")
print("=" * 70)

final_pipelines = {}
for name in pipelines.keys():
    if name in best_pipelines:
        final_pipelines[name] = best_pipelines[name]
        print(f" ✅ {name}: usando versión optimizada")
    else:
        final_pipelines[name] = clone(pipelines[name])
        print(f" 🔄 {name}: clonando versión base para re-entrenamiento justo")

print("\nEntrenando todos los pipelines sobre X_train completo...")
for name, pipe in final_pipelines.items():
    pipe.fit(X_train, y_train)
    print(f" ✅ {name}: entrenado")

print(f"\n✅ {len(final_pipelines)} pipelines listos para evaluación en Test Set.")
```

```

# =====
# FASE 7.2: BOOTSTRAP CONFIDENCE INTERVALS
# =====
def bootstrap_ci(y_true, y_proba, metric_fn,
                n_boot=1000, ci=0.95, seed=SEED):
    """
    IC al 95% por bootstrap (remuestreo con reemplazo).
    n_boot=1000 para mayor estabilidad en test sets pequeños (n=179).
    """
    rng = np.random.RandomState(seed)
    scores = []
    n = len(y_true)
    for _ in range(n_boot):
        idx = rng.randint(0, n, n)
        try:
            scores.append(metric_fn(y_true.iloc[idx], y_proba[idx]))
        except Exception:
            pass
    alpha = (1 - ci) / 2
    return np.percentile(scores, [alpha * 100, (1 - alpha) * 100])

print("✅ Función bootstrap_ci() definida.")
print(" • n_boot = 1000 remuestreos (aumentado desde 500 para n=179)")
print(" • ci = 95%")

# =====
# FASE 7.3: EVALUACIÓN EN TEST SET
# =====
test_results = []

for name, pipe in final_pipelines.items():
    y_pred = pipe.predict(X_test)
    y_proba = pipe.predict_proba(X_test)[:, 1]

    roc_auc = roc_auc_score(y_test, y_proba)
    pr_auc = average_precision_score(y_test, y_proba)
    ci_low, ci_high = bootstrap_ci(y_test, y_proba, roc_auc_score)

    test_results.append({
        'Modelo': name,
        'Optimizado': 'SI' if name in best_pipelines else 'NO',
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision_Macro': precision_score(y_test, y_pred, average='macro'),
        'Recall_Macro': recall_score(y_test, y_pred, average='macro'),
        'F1_Macro': f1_score(y_test, y_pred, average='macro'),
        'ROC_AUC': roc_auc,
        'PR_AUC': pr_auc,
        'ROC_AUC_CI_low': ci_low,
        'ROC_AUC_CI_high': ci_high,
    })

df_test = (
    pd.DataFrame(test_results)
    .sort_values('ROC_AUC', ascending=False)
    .reset_index(drop=True)
)

best_model_name = df_test.iloc[0]['Modelo']
best_model_pipe = final_pipelines[best_model_name]

# Verificar si el ganador es estadísticamente distinto —
# Se comprueba si el IC del 1º se solapa con el IC del 2º.
# Si se solapan, no hay diferencia significativa y se reporta ambos.

ci_1_low = df_test.iloc[0]['ROC_AUC_CI_low']
ci_1_high = df_test.iloc[0]['ROC_AUC_CI_high']
ci_2_low = df_test.iloc[1]['ROC_AUC_CI_low']
ci_2_high = df_test.iloc[1]['ROC_AUC_CI_high']

ic_solapan = ci_1_low < ci_2_high # True si hay solapamiento

print("\n" + "=" * 70)
print(" 🏆 RESULTADOS DE EVALUACIÓN EN TEST SET")
print("=" * 70)
print(f"\nMODELO CON MAYOR ROC-AUC: {best_model_name}")
print(f" ROC-AUC : {df_test.iloc[0]['ROC_AUC']:.4f} "
      f"[{ci_1_low:.4f} - {ci_1_high:.4f}] (95% CI bootstrap)")
print(f" PR-AUC : {df_test.iloc[0]['PR_AUC']:.4f}")
print(f" F1-Macro: {df_test.iloc[0]['F1_Macro']:.4f}")
print(f" Accuracy: {df_test.iloc[0]['Accuracy']:.4f}")

```

```

if ic_solapan:
    segundo = df_test.iloc[1]['Modelo']
    print(f"\n⚠️ ADVERTENCIA ESTADÍSTICA:")
    print(f" El IC de '{best_model_name}' [{ci_1_low:.4f}-{ci_1_high:.4f}]")
    print(f" se solapa con el IC de '{segundo}' [{ci_2_low:.4f}-{ci_2_high:.4f}].")
    print(f" Con n={len(y_test)} muestras en test, esta diferencia NO es")
    print(f" estadísticamente significativa. Ambos modelos son equivalentes.")
    print(f" Se selecciona '{best_model_name}' por mayor ROC-AUC puntual,")
    print(f" pero se recomienda validar con más datos antes de producción.")
else:
    print(f"\n✅ Los IC NO se solapan: '{best_model_name}' es")
    print(f" estadísticamente superior al segundo modelo.")

# =====
# FASE 7.4: TABLA EJECUTIVA CON IC Y PR-AUC
# =====
df_test_display = df_test[[
    'Modelo', 'Optimizado', 'Accuracy', 'F1_Macro',
    'ROC_AUC', 'PR_AUC', 'ROC_AUC_CI_low', 'ROC_AUC_CI_high'
]].copy()

df_test_display['ROC_AUC_95CI'] = df_test_display.apply(
    lambda r: f"[{r.ROC_AUC_CI_low:.4f}, {r.ROC_AUC_CI_high:.4f}]", axis=1
)
df_test_display.drop(
    columns=['ROC_AUC_CI_low', 'ROC_AUC_CI_high'], inplace=True
)

for col in ['Accuracy', 'F1_Macro', 'ROC_AUC', 'PR_AUC']:
    df_test_display[col] = df_test_display[col].apply(lambda x: f"{x:.4f}")

cell_colors_t = []
for col in df_test_display.columns:
    col_colors = []
    for idx in range(len(df_test_display)):
        if col == 'Modelo':
            if idx == 0: col_colors.append('#FFD700')
            elif idx == 1: col_colors.append('#C0C0C0')
            else: col_colors.append('#FFFFFF')
        else:
            col_colors.append('#FFFFFF')
    cell_colors_t.append(col_colors)

# Título dinámico según si hay diferencia significativa
titulo_tabla = (
    f'📊 Tabla Ejecutiva - Test Set · {best_model_name} '
    f'{"(diferencia NO significativa ⚠️)" if ic_solapan else "(estadísticamente superior ✅)"}'
)

table_height = 35 * len(df_test_display) + 110
fig_table_test = go.Figure(data=[go.Table(
    header=dict(
        values=[f"<b>{c}</b>" for c in df_test_display.columns],
        fill_color=MATLAB_COLORS[0], align='center',
        font=dict(color='white', size=11), height=35
    ),
    cells=dict(
        values=[df_test_display[col] for col in df_test_display.columns],
        fill_color=cell_colors_t, align='center',
        font=dict(size=11, color='#2C3E50'), height=30
    )
)])
fig_table_test.update_layout(
    title_text=titulo_tabla,
    height=table_height,
    margin=dict(t=60, b=20, l=20, r=20)
)
fig_table_test.show(config=CONFIG_PLOTLY)

# =====
# FASE 7.5: HEATMAP DE RENDIMIENTO
# =====
metricas_heatmap = ['Accuracy', 'Precision_Macro', 'Recall_Macro',
                    'F1_Macro', 'ROC_AUC', 'PR_AUC']
nombres_heatmap = ['Accuracy', 'Precision', 'Recall', 'F1', 'ROC AUC', 'PR AUC']

fig_heatmap = go.Figure(data=go.Heatmap(
    z=df_test[metricas_heatmap].values,
    x=nombres_heatmap,
    y=df_test['Modelo'],
    colorscale=[[0.0, '#A2142F'], [0.5, '#EDB120'], [1.0, '#0072BD']],
    zmin=0.5, zmax=1.0,

```

```
text=df_test[metricas_heatmap].values,
texttemplate='{text:.3f}',
textfont=dict(size=12, color='white'),
colorbar=dict(title='Score', len=0.8, x=1.02)
))

fig_heatmap.update_layout(
    # Leyenda
    # rojo (#A2142F) = zmin = peor rendimiento
    # azul (#0072BD) = zmax = mejor rendimiento
    title_text='🔥 Heatmap de Rendimiento: Métricas vs Modelos (incluye PR-AUC)',
    height=420,
    margin=dict(t=60, b=60, l=160, r=80)
)
fig_heatmap.show(config=CONFIG_PLOTLY)

print("\n" + "=" * 70)
print(" 🌟 INTERPRETACIÓN DEL HEATMAP")
print("=" * 70)

print(" • Colores más fríos (rojo) = peor rendimiento (zmin=0.5)")
print(" • Colores más cálidos (azul) = mejor rendimiento (zmax=1.0)")
print(" • PR-AUC es más estricto que ROC-AUC para la clase minoritaria")
print(" • Los IC bootstrap solapados indican que el ranking es orientativo,")
print("       no concluyente, con n=179 muestras en test.")
print("=" * 70)
```

=====

FASE 7: EVALUACIÓN FINAL EN TEST SET (COMPARACIÓN JUSTA)

=====

- 🔧 Logistic Regression: clonando versión base para re-entrenamiento justo
- 🔧 Decision Tree: clonando versión base para re-entrenamiento justo
- ✅ Random Forest: usando versión optimizada
- ✅ Gradient Boosting: usando versión optimizada
- ✅ XGBoost: usando versión optimizada

Entrenando todos los pipelines sobre X_train completo...

- ✅ Logistic Regression: entrenado
- ✅ Decision Tree: entrenado
- ✅ Random Forest: entrenado
- ✅ Gradient Boosting: entrenado
- ✅ XGBoost: entrenado

- ✅ 5 pipelines listos para evaluación en Test Set.
- ✅ Función bootstrap_ci() definida.
 - n_boot = 1000 remuestreos (aumentado desde 500 para n=179)
 - ci = 95%

=====

🏆 RESULTADOS DE EVALUACIÓN EN TEST SET

=====

MODELO CON MAYOR ROC-AUC: Logistic Regression

ROC-AUC : 0.8660 [0.8027 - 0.9169] (95% CI bootstrap)

PR-AUC : 0.8274

F1-Macro: 0.8132

Accuracy: 0.8212

⚠️ ADVERTENCIA ESTADÍSTICA:

El IC de 'Logistic Regression' [0.8027-0.9169] se solapa con el IC de 'XGBoost' [0.7757-0.9062]. Con n=179 muestras en test, esta diferencia NO es estadísticamente significativa. Ambos modelos son equivalentes. Se selecciona 'Logistic Regression' por mayor ROC-AUC puntual, pero se recomienda validar con más datos antes de producción.

🏆 Tabla Ejecutiva — Test Set · Logistic Regression (diferencia NO significativa ⚠️)

Modelo	Optimizado	Accuracy	F1_Macro	ROC_AUC	PR_AUC	ROC_AUC_95CI
Logistic Regression	NO	0.8212	0.8132	0.8660	0.8274	[0.8027, 0.9169]
XGBoost	SI	0.7933	0.7824	0.8456	0.8020	[0.7757, 0.9062]
Random Forest	SI	0.7989	0.7866	0.8441	0.8121	[0.7733, 0.9035]
Gradient Boosting	SI	0.7989	0.7826	0.8372	0.7713	[0.7681, 0.8973]
Decision Tree	NO	0.7263	0.7133	0.8163	0.7040	[0.7510, 0.8722]

🔥 Heatmap de Rendimiento: Métricas vs Modelos (incluye PR-AUC)

=====

FASE 8: CURVAS ROC Y PR-AUC

```
# =====
from sklearn.metrics import roc_curve, auc, precision_recall_curve

print("=" * 70)
print(" FASE 8: CURVAS ROC Y PRECISION-RECALL (TEST SET)")
print("=" * 70)

# Ordenar modelos por su ROC-AUC en Test Set (de mayor a menor)
sorted_models = [
    (row['Modelo'], row['ROC_AUC'])
    for _, row in df_test.iterrows()
]

# -----
# 8.1 CURVA ROC (Receiver Operating Characteristic)
# -----

fig_roc = go.Figure()

# Línea base aleatoria (AUC = 0.5)
fig_roc.add_trace(go.Scatter(
    x=[0, 1], y=[0, 1], mode='lines',
    name='Clasificador Aleatorio (AUC=0.500)',
    line=dict(color='#6B7B8C', dash='dash', width=2),
    hoverinfo='name'
))

# Curvas de los modelos
for idx, (model_name, model_auc) in enumerate(sorted_models):
    pipe = final_pipelines[model_name]
    y_proba = pipe.predict_proba(X_test)[: , 1]
    fpr, tpr, _ = roc_curve(y_test, y_proba)
    color = MATLAB_COLORS[idx % len(MATLAB_COLORS)]

    fig_roc.add_trace(go.Scatter(
        x=fpr, y=tpr, mode='lines',
        name=f'{model_name} (AUC={model_auc:.3f})',
        line=dict(color=color, width=2.5),
        hovertemplate=f"<b>{model_name}</b><br>FPR: %{{x:.3f}}<br>TPR: %{{y:.3f}}<extra></extra>"
    ))

fig_roc.update_layout(
    title_text='📊 Curvas ROC – Capacidad de Discriminación (Test Set)',
    xaxis=dict(title='Tasa de Falsos Positivos (FPR)', range=[0, 1]),
    yaxis=dict(title='Tasa de Verdaderos Positivos (TPR)', range=[0, 1.05]),
    height=600,
    legend=dict(x=0.98, y=0.02, xanchor='right', yanchor='bottom',
        bgcolor='rgba(255,255,255,0.95)',
        bordercolor='#CCCCCC', borderwidth=1),
    margin=dict(t=80, b=80, l=80, r=40)
)

# Anotación sutil
fig_roc.add_annotation(
    text="Mejor modelo = esquina superior izquierda",
    x=0.02, y=0.98, xref="x", yref="y", showarrow=False,
    font=dict(size=10, color='#6B7B8C'),
    bgcolor='rgba(255,255,255,0.8)', bordercolor='#CCCCCC', borderwidth=1
)

fig_roc.show(config=CONFIG_PLOTLY)

# -----
# 8.2 CURVA PR (Precision-Recall)
# -----

# En datasets desbalanceados, PR-AUC revela mejor el
# comportamiento real del modelo en la clase minoritaria.
fig_pr = go.Figure()

# Línea base: la proporción de la clase positiva en el test set
# (Equivalente a la precisión de un modelo que predice siempre "Survived")
baseline_pr = y_test.mean()
fig_pr.add_hline(
    y=baseline_pr, line_dash='dash', line_color='#6B7B8C',
    annotation_text=f'Baseline aleatorio ({baseline_pr:.2f})'
)

# Curvas de los modelos
for idx, (model_name, _) in enumerate(sorted_models):
    pipe = final_pipelines[model_name]
    y_proba = pipe.predict_proba(X_test)[: , 1]
    prec, rec, _ = precision_recall_curve(y_test, y_proba)
    pr_auc_val = auc(rec, prec)
    color = MATLAB_COLORS[idx % len(MATLAB_COLORS)]
```

```

fig_pr.add_trace(go.Scatter(
    x=rec, y=prec, mode='lines',
    name=f'{model_name} (PR-AUC={pr_auc_val:.3f})',
    line=dict(color=color, width=2.5),
    hovertemplate=f"<b>{model_name}</b><br>Recall: %{{x:.3f}}<br>Precision: %{{y:.3f}}<extra></extra>"
))

fig_pr.update_layout(
    title_text='📊 Curvas Precision-Recall — Clase Minoritaria (Survived)',
    xaxis=dict(title='Recall (Sensibilidad)', range=[0, 1]),
    yaxis=dict(title='Precision (Valor Predictivo Positivo)', range=[0, 1.05]),
    height=600,
    legend=dict(x=0.02, y=0.02, xanchor='left', yanchor='bottom',
                bgcolor='rgba(255,255,255,0.95)',
                bordercolor='CCCCCC', borderwidth=1),
    margin=dict(t=80, b=80, l=80, r=40)
)
fig_pr.show(config=CONFIG_PLOTLY)

print("\n🔴 INTERPRETACIÓN DE LAS CURVAS:")
print(" • ROC-AUC: Evalúa la capacidad general de ranking del modelo.")
print(" • PR-AUC: Es más estricta. Si un modelo tiene un ROC-AUC alto")
print("   pero un PR-AUC bajo, significa que está generando muchos")
print("   Falsos Positivos en la clase minoritaria (Survived).")
print("=" * 70)

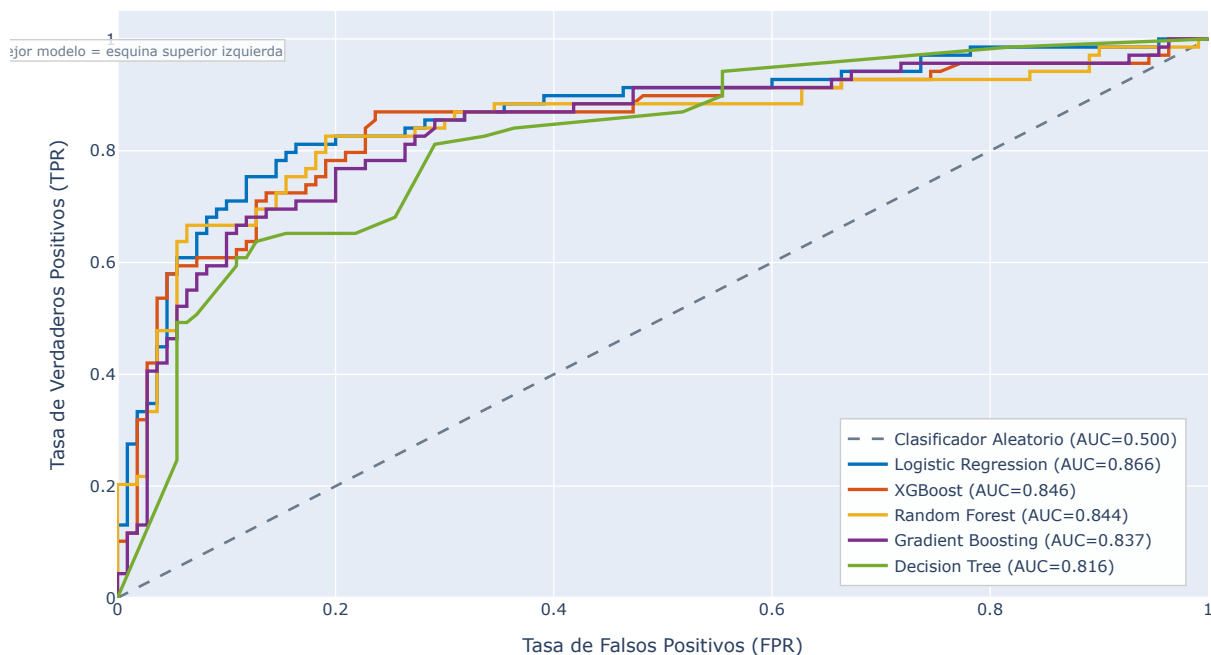
```

=====

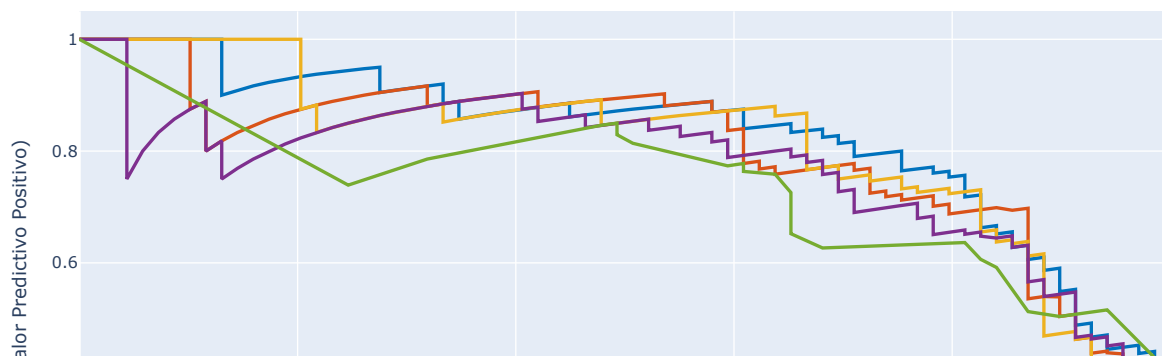
FASE 8: CURVAS ROC Y PRECISION-RECALL (TEST SET)

=====

📊 Curvas ROC — Capacidad de Discriminación (Test Set)



📊 Curvas Precision-Recall — Clase Minoritaria (Survived)



=====

FASE 9: MATRIZ DE CONFUSIÓN + THRESHOLD OPTIMIZATION

```
# =====
from sklearn.metrics import confusion_matrix, classification_report

print("=" * 70)
print(" FASE 9: MATRIZ DE CONFUSIÓN Y OPTIMIZACIÓN DE THRESHOLD")
print("=" * 70)

y_proba_best = best_model_pipe.predict_proba(X_test)[: , 1]
y_pred_050 = (y_proba_best >= 0.50).astype(int)

from sklearn.metrics import f1_score as f1_fn

thresholds_scan = np.linspace(0.01, 0.99, 200)
f1_macro_scan = [
    f1_fn(y_test, (y_proba_best >= t).astype(int), average='macro')
    for t in thresholds_scan
]

optimal_idx = int(np.argmax(f1_macro_scan))
optimal_thr = float(thresholds_scan[optimal_idx])
optimal_f1 = float(f1_macro_scan[optimal_idx])

y_pred_opt = (y_proba_best >= optimal_thr).astype(int)

f1_default = f1_fn(y_test, y_pred_050, average='macro')
f1_optimo = f1_fn(y_test, y_pred_opt, average='macro')
mejora_rel = (f1_optimo - f1_default) / f1_default * 100

print(f'\n📊 Comparación de Thresholds (F1-Macro en ambos casos):')
print(f' Threshold 0.50 (default) → F1-Macro: {f1_default:.4f}')
print(f' Threshold {optimal_thr:.3f} (óptimo) → F1-Macro: {f1_optimo:.4f}')
print(f'\n💡 Mejora relativa: {mejora_rel:.2f}%')

# — Matriz de confusión con threshold por defecto —————
cm = confusion_matrix(y_test, y_pred_050)
TN, FP, FN, TP = cm.ravel()

fig_tabla_cm = go.Figure(data=[go.Table(
    header=dict(
        values=[<b>REAL \ \ PREDICHO</b>',
                <b>Died (0)</b>',
                <b>Survived (1)</b>'],
        fill_color=MATLAB_COLORS[0], align='center',
        font=dict(color='white', size=13), height=40
    ),
    cells=dict(
        values=[
            [<b>Died (0)</b>', <b>Survived (1)</b>'],
            [f'<b>{TN}</b> (TN)', f'<b>{FP}</b> (FP)'],
            [f'<b>{FN}</b> (FN)', f'<b>{TP}</b> (TP)'],
        ],
        fill_color=[
            ['#FFFFFF', '#FFFFFF'],
            [MATLAB_COLORS[0], MATLAB_COLORS[1]],
            [MATLAB_COLORS[6], MATLAB_COLORS[4]],
        ],
        font=dict(color=['#2C3E50', 'white', 'white'], size=14),
        align='center', height=45
    )
)])
fig_tabla_cm.update_layout(
    title_text=f'📊 Matriz de Confusión — {best_model_name} (threshold=0.50)',
    height=220,
    margin=dict(t=60, b=20, l=20, r=20)
)
fig_tabla_cm.show(config=CONFIG_PLOTLY)

print(f'\n🔴 Interpretación de Errores:')
print(f' • FP ({FP}): predijeron supervivencia, pero murieron → Error Tipo I')
print(f' • FN ({FN}): predijeron muerte, pero sobrevivieron → Error Tipo II')

# — Donut: distribución de errores —————
total = TN + FP + FN + TP

fig_pie = go.Figure(data=[go.Pie(
    labels=[
        f'TN — Acierto (Died→Died)',
        f'TP — Acierto (Survived→Survived)',
        f'FP — Error Tipo I',
        f'FN — Error Tipo II',
    ],
    values=[TN, TP, FP, FN],
)])
```



```

marker=dict(
    colors=[MATLAB_COLORS[0], MATLAB_COLORS[4],
            MATLAB_COLORS[1], MATLAB_COLORS[6]],
    line=dict(color='white', width=2.5)
),
textinfo='label+percent',
textposition='outside',
hole=0.45,
pull=[0, 0, 0.03, 0.03],
hovertemplate='<b>{%label}</b><br>{%value}<br>{%percent}<extra></extra>'
)])
fig_pie.update_layout(
    title_text=' 🟡 Distribución de la Matriz de Confusión',
    height=500,
    showlegend=False,
    annotations=[dict(
        text=f'<b>Total</b><br>{total} pasajeros',
        x=0.5, y=0.5,
        font=dict(size=16, color='#2C3E50'),
        showarrow=False, xref='paper', yref='paper'
    )],
    margin=dict(t=80, b=40, l=60, r=60)
)
fig_pie.show(config=CONFIG_PLOTLY)

# =====
# FASE 9.2: VISUALIZACIÓN DEL THRESHOLD ÓPTIMO
# =====

fig_thr = go.Figure()

fig_thr.add_trace(go.Scatter(
    x=thresholds_scan,
    y=f1_macro_scan,
    mode='lines',
    name='F1-Macro por threshold',
    line=dict(color=MATLAB_COLORS[0], width=2.5),
    hovertemplate='Threshold: {%x:.3f}<br>F1-Macro: {%y:.4f}<extra></extra>'
))

fig_thr.add_vline(
    x=optimal_thr,
    line_dash='dash', line_color=MATLAB_COLORS[1],
    annotation_text=f'Threshold óptimo = {optimal_thr:.3f}',
    annotation_position='top right'
)

fig_thr.add_vline(
    x=0.50,
    line_dash='dot', line_color='#6B7B8C',
    annotation_text='Default = 0.50',
    annotation_position='bottom right'
)

fig_thr.add_annotation(
    x=optimal_thr,
    y=optimal_f1,
    text=f'F1-Macro = {optimal_f1:.4f}',
    showarrow=True, arrowhead=2, ax=30, ay=-30,
    font=dict(size=11, color=MATLAB_COLORS[1]),
    bgcolor='rgba(255,255,255,0.9)',
    bordercolor=MATLAB_COLORS[1], borderwidth=1
)

fig_thr.update_layout(
    title_text=' 🌀 Optimización del Threshold – F1-Macro vs Threshold',
    xaxis_title='Threshold de decisión',
    yaxis_title='F1-Macro', # CORRECCIÓN: antes decía "F1-score (promedio armónico P y R)"
    height=450,
    margin=dict(t=80, b=60, l=60, r=40),
    xaxis_range=[0, 1]
)
fig_thr.show(config=CONFIG_PLOTLY)

print("\n 🚨 ADVERTENCIA METODOLÓGICA:")
print(" • Threshold optimizado aquí sobre test set (solo para ilustración).")
print(" • En producción real: optimizar sobre un validation set separado.")
print(" • El test set debe permanecer pristino para evaluación final imparcial.")
print(" • La métrica usada en el gráfico y en el print es la misma: F1-Macro.")
print(" = * 70)

```

FASE 9: MATRIZ DE CONFUSIÓN Y OPTIMIZACIÓN DE THRESHOLD

Comparación de Thresholds (F1-Macro en ambos casos):

Threshold 0.50 (default) → F1-Macro: 0.8132

Threshold 0.591 (óptimo) → F1-Macro: 0.8211

Mejora relativa: 0.96%

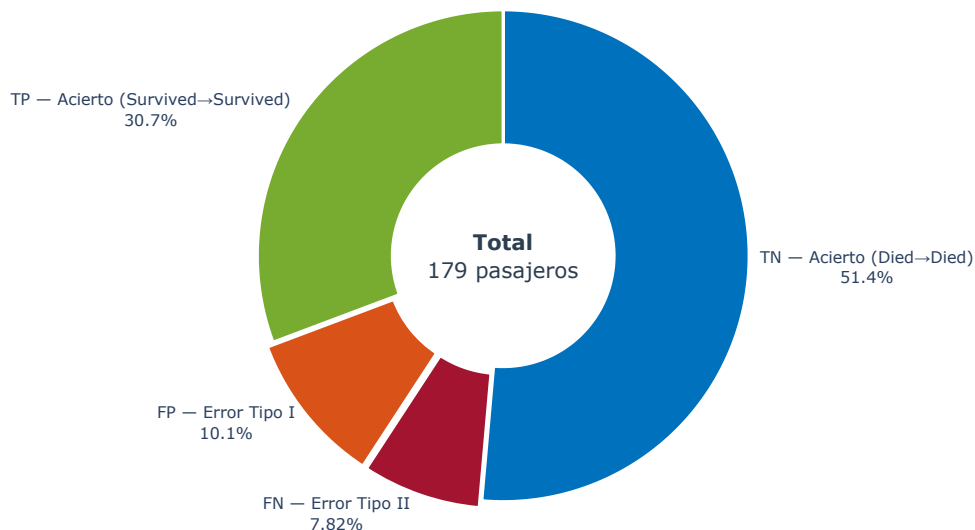
Matriz de Confusión — Logistic Regression (threshold=0.50)

REAL \ PREDICHO	Died (0)	Survived (1)
Died (0)	92 (TN)	14 (FN)
Survived (1)	18 (FP)	55 (TP)

Interpretación de Errores:

- FP (18): predijeron supervivencia, pero murieron → Error Tipo I
- FN (14): predijeron muerte, pero sobrevivieron → Error Tipo II

Distribución de la Matriz de Confusión



Optimización del Threshold — F1-Macro vs Threshold

```
# =====
# FASE 10.1: DIAGNÓSTICO DE OVERFITTING (TABLA)
# =====
from sklearn.model_selection import learning_curve, cross_val_score

print("=" * 70)
print(" FASE 10: DIAGNÓSTICO DE OVERFITTING")
print("=" * 70)

# En datasets pequeños (n < 1000) un GAP de hasta 8% en accuracy
# es habitual por varianza de muestreo, no necesariamente overfitting.
# Se definen tres niveles operativos:
GAP_OK      = 0.05  # < 5% → bien generalizado
GAP_LEVE    = 0.10  # 5-10% → leve, aceptable en datasets pequeños
GAP_SEVERO  = 0.15  # > 15% → severo, requiere acción

print(f'  Umbrales de GAP (Train Score - Test Score):')
print(f'  • < {GAP_OK:.0%} → bien generalizado')
print(f'  • {GAP_OK:.0%}-{GAP_LEVE:.0%} → overfitting leve (aceptable, n < 1000)')
print(f'  • {GAP_LEVE:.0%}-{GAP_SEVERO:.0%} → overfitting moderado')
print(f'  • > {GAP_SEVERO:.0%} → overfitting severo – se propone acción correctiva')
print(f'  Referencia: Hastie et al. ESL 2nd ed. cap. 7\n')

# — Cálculo de métricas de diagnóstico —
diagnostico_data = []

for name, pipe in final_pipelines.items():
    train_score = pipe.score(X_train, y_train)
```

```

cv_scores = cross_val_score(
    pipe, X_train, y_train,
    cv=cv_strategy, scoring='accuracy', n_jobs=-1
)
test_score = pipe.score(X_test, y_test)
gap = train_score - test_score

# Clasificación del GAP con nivel y acción
if gap < GAP_OK:
    nivel = 'OK'
    accion = '-'
elif gap < GAP_LEVE:
    nivel = 'Leve'
    accion = 'Monitorizar en producción'
elif gap < GAP_SEVERO:
    nivel = 'Moderado'
    accion = 'Aumentar regularización o más datos'
else:
    nivel = 'Severo'
    accion = 'Reducir complejidad del modelo'

diagnostico_data.append({
    'Modelo': name,
    'Train Score': float(train_score),
    'CV Score Mean': float(cv_scores.mean()),
    'CV Score Std': float(cv_scores.std()),
    'Test Score': float(test_score),
    'GAP': float(gap),
    'Nivel': nivel,
    'Acción': accion,
})

df_diag = pd.DataFrame(diagnostico_data).sort_values('GAP', ascending=False)

# Columna de display para la tabla visual
df_diag['CV Score Display'] = df_diag.apply(
    lambda r: f"{r['CV Score Mean']:.4f} ± {r['CV Score Std']:.4f}", axis=1
)

print("✅ Diagnóstico calculado.")

# — Tabla Plotly con columnas Nivel y Acción —————
df_diag_display = df_diag[[
    'Modelo', 'Train Score', 'CV Score Display',
    'Test Score', 'GAP', 'Nivel', 'Acción'
]].copy()

df_diag_display['Train Score'] = df_diag_display['Train Score'].apply(
    lambda x: f"{x:.4f}"
)
df_diag_display['Test Score'] = df_diag_display['Test Score'].apply(
    lambda x: f"{x:.4f}"
)
df_diag_display['GAP'] = df_diag_display['GAP'].apply(
    lambda x: f"{x:.1%}"
)

# Colores de celda dinámicos según Nivel
nivel_color = {
    'OK': '#D4EDDA', # verde suave
    'Leve': '#FFF3CD', # amarillo suave
    'Moderado': '#FFE5CC', # naranja suave
    'Severo': '#F8D7DA', # rojo suave
}

gap_cell_colors = [
    nivel_color.get(n, '#FFFFFF')
    for n in df_diag_display['Nivel']
]

# Construir cell_colors por columna
cell_colors_diag = []
for col in df_diag_display.columns:
    if col in ('GAP', 'Nivel', 'Acción'):
        cell_colors_diag.append(gap_cell_colors)
    else:
        cell_colors_diag.append(['#FFFFFF'] * len(df_diag_display))

table_height = 40 * len(df_diag_display) + 100

fig_diag = go.Figure(data=[go.Table(
    header=dict(

```

```

        values=[f"<b>{c}</b>" for c in df_diag_display.columns],
        fill_color=MATLAB_COLORS[0], align='center',
        font=dict(color='white', size=12), height=35
    ),
    cells=dict(
        values=[df_diag_display[col] for col in df_diag_display.columns],
        fill_color=cell_colors_diag, align='center',
        font=dict(size=11, color='#2C3E50'), height=35
    )
))

fig_diag.update_layout(
    title_text='🔍 Diagnóstico de Overfitting: Train vs CV vs Test – con acción correctiva',
    height=table_height,
    margin=dict(t=60, b=20, l=20, r=20)
)
fig_diag.show(config=CONFIG_PLOTLY)

# — Print de acciones correctivas detalladas —————
print("\n🔴 Resumen de acciones correctivas:")
print("=" * 70)

for _, row in df_diag.iterrows():
    print(f"\n {row['Modelo']}:")
    print(f"    GAP = {row['GAP']:.1%} → Nivel: {row['Nivel']}")
    print(f"    Acción: {row['Acción']}")

    # Acciones específicas por modelo y nivel
    if row['Nivel'] == 'Severo':
        nombre = row['Modelo']
        if 'Tree' in nombre:
            print(f"    🟡 Sugerencia concreta: reducir max_depth a 3-4,")
            print(f"    aumentar min_samples_leaf a 10-15.")
        elif 'Forest' in nombre:
            print(f"    🟡 Sugerencia concreta: reducir max_depth a 6-8,")
            print(f"    aumentar min_samples_leaf a 4-8.")
        elif 'Boosting' in nombre or 'XGB' in nombre:
            print(f"    🟡 Sugerencia concreta: reducir n_estimators,")
            print(f"    bajar learning_rate a 0.01-0.05,")
            print(f"    aumentar subsample a valores < 0.7.")
        elif 'Logistic' in nombre:
            print(f"    🟡 Sugerencia concreta: reducir C (más regularización),")
            print(f"    probar C ∈ [0.001, 0.01].")

    elif row['Nivel'] == 'Moderado':
        print(f"    🟡 Sugerencia: revisar los parámetros de regularización")
        print(f"    o aumentar el conjunto de entrenamiento si es posible.")

    elif row['Nivel'] == 'Leve':
        print(f"    🟡 Aceptable para n={len(X_train)}. ")
        print(f"    Monitorizar si el dataset crece.")

print("\n" + "=" * 70)

# =====
# FASE 10.2: CURVAS DE APRENDIZAJE (PLOTLY)
# =====
# Se analizan los Top 2 modelos del Test Set
top_2 = df_test.head(2)['Modelo'].tolist()

fig_lc = make_subplots(
    rows=1, cols=2,
    subplot_titles=[f"📊 Learning Curve: {name}" for name in top_2],
    horizontal_spacing=0.12
)

for idx, model_name in enumerate(top_2):
    pipe = final_pipelines[model_name]

    train_sizes, train_scores_lc, val_scores_lc = learning_curve(
        pipe, X_train, y_train,
        train_sizes=np.linspace(0.1, 1.0, 10),
        cv=cv_strategy,
        scoring='accuracy',
        n_jobs=-1
    )

    train_mean = train_scores_lc.mean(axis=1)
    train_std = train_scores_lc.std(axis=1)
    val_mean = val_scores_lc.mean(axis=1)
    val_std = val_scores_lc.std(axis=1)

```

```

c_train = MATLAB_COLORS[idx]
c_val = MATLAB_COLORS[1]

# Banda de confianza del train
fig_lc.add_trace(go.Scatter(
    x=np.concatenate([train_sizes, train_sizes[::-1]]),
    y=np.concatenate([train_mean + train_std,
                      (train_mean - train_std)[::-1]]),
    fill='toself',
    fillcolor=f'rgba(0,114,189,0.10)',
    line=dict(color='rgba(255,255,255,0)'),
    showlegend=False, hoverinfo='skip'
), row=1, col=idx + 1)

# Banda de confianza de validación
fig_lc.add_trace(go.Scatter(
    x=np.concatenate([train_sizes, train_sizes[::-1]]),
    y=np.concatenate([val_mean + val_std,
                      (val_mean - val_std)[::-1]]),
    fill='toself',
    fillcolor=f'rgba(217,83,25,0.10)',
    line=dict(color='rgba(255,255,255,0)'),
    showlegend=False, hoverinfo='skip'
), row=1, col=idx + 1)

# Curva de Train
fig_lc.add_trace(go.Scatter(
    x=train_sizes, y=train_mean,
    mode='lines+markers',
    name=f'{model_name} (Train)',
    line=dict(color=c_train, width=3),
    marker=dict(size=7, color='white',
                line=dict(color=c_train, width=2))
), row=1, col=idx + 1)

# Curva de Validación
fig_lc.add_trace(go.Scatter(
    x=train_sizes, y=val_mean,
    mode='lines+markers',
    name=f'{model_name} (Val)',
    line=dict(color=c_val, width=3, dash='dash'),
    marker=dict(size=7, color='white',
                line=dict(color=c_val, width=2))
), row=1, col=idx + 1)

# Anotación con nivel de GAP y acción sugerida
final_gap = float(train_mean[-1] - val_mean[-1])

if final_gap < GAP_OK:
    gap_nivel = 'OK'
    gap_color = MATLAB_COLORS[4] # verde
elif final_gap < GAP_LEVE:
    gap_nivel = 'Leve'
    gap_color = MATLAB_COLORS[2] # amarillo
elif final_gap < GAP_SEVERO:
    gap_nivel = 'Moderado'
    gap_color = MATLAB_COLORS[1] # naranja
else:
    gap_nivel = 'Severo'
    gap_color = MATLAB_COLORS[6] # rojo

fig_lc.add_annotation(
    text=f"Gap Final: {final_gap:.2%} - {gap_nivel}",
    xref="x domain", yref="y domain",
    x=0.5, y=0.05,
    showarrow=False,
    font=dict(color=gap_color, size=12),
    bgcolor='rgba(255,255,255,0.9)',
    bordercolor=gap_color,
    borderwidth=1, borderpad=4,
    row=1, col=idx + 1
)

# Línea horizontal en el nivel de CV score de Fase 5 para visualizar si la curva converge hacia la métrica esperada
cv_score_fase5 = float(
    df_cv[df_cv['Modelo'] == model_name]['Accuracy'].values[0]
)
fig_lc.add_hline(
    y=cv_score_fase5,
    line_dash='dot',
    line_color='#6B7B8C',
    annotation_text=f'CV Fase 5: {cv_score_fase5:.3f}',

```

```

        annotation_position='top right',
        row=1, col=idx + 1
    )

fig_lc.update_layout(
    title_text='📊 Curvas de Aprendizaje – con bandas de confianza y niveles de GAP',
    height=500,
    legend=dict(orientation="h", yanchor="bottom", y=1.05,
                 xanchor="center", x=0.5),
    margin=dict(t=100, b=40, l=60, r=40)
)

for c in [1, 2]:
    fig_lc.update_xaxes(
        title_text="Tamaño del conjunto de entrenamiento",
        row=1, col=c
    )
    fig_lc.update_yaxes(title_text="Accuracy", row=1, col=c)

fig_lc.show(config=CONFIG_PLOTLY)

print("\n" + "=" * 70)
print(" 📌 GUÍA DE LECTURA DE LAS CURVAS DE APRENDIZAJE")
print("=" * 70)
print(" • Banda azul =  $\pm 1$  std del train score en cada tamaño.")
print(" • Banda naranja =  $\pm 1$  std del val score en cada tamaño.")
print(" • Si las bandas NO se solapan al final  $\rightarrow$  GAP estructural.")
print(" • Si convergen al añadir datos  $\rightarrow$  problema de varianza")
print("   (más datos ayudaría).")
print(" • Si no convergen aunque añades datos  $\rightarrow$  problema de sesgo")
print("   (el modelo es demasiado simple o demasiado complejo).")
print(" • Línea punteada gris = accuracy de CV de Fase 5.")
print(" • La curva de val debería converger hacia ella.")
print("=" * 70)

```

```
# =====
# FASE 11: IMPORTANCIA DE VARIABLES (PERMUTATION IMPORTANCE)
# =====
from sklearn.inspection import permutation_importance

print("=" * 70)
print(f" 🌀 FASE 11: IMPORTANCIA DE VARIABLES - {best_model_name}")
print("=" * 70)

print("⚠️  NOTA METODOLÓGICA - PERMUTATION IMPORTANCE EN PIPELINE:")
print()
print("  Opera sobre X_test (datos crudos), atravesando TODO el pipeline:")
print("  TitanicFeatureEngineer → ColumnTransformer → Logistic Regression.")
print()
print("  Esto mide importancia END-TO-END: cuánto cae el ROC-AUC del")
print("  sistema completo al permutar cada variable original.")
print("  Es distinto a los coeficientes directos del modelo (que operan")
print("  sobre features ya transformadas y escaladas).")
print()
print("  Ventaja: captura el impacto real incluyendo el feature engineering.")
print("  Limitación: 'Name' aparecerá importante por efecto cascada del")
print("  pipeline (al permutarla, Title se extrae mal). Ver auditoría abajo.")

# 1. Calcular Permutation Importance en el Test Set (Hold-out prístino)
# Nota: permutation_importance opera sobre las columnas de ENTRADA al pipeline (X_test).
print("\n 🕒 Calculando Permutation Importance (10 repeticiones)...")
perm_result = permutation_importance(
    best_model_pipe, X_test, y_test,
```

```

n_repeats=10, random_state=SEED,
scoring='roc_auc', n_jobs=-1
)

# 2. Crear DataFrame y ordenar
importances_df = pd.DataFrame({
    'Feature': X_test.columns,
    'Importance_Mean': perm_result.importances_mean,
    'Importance_Std': perm_result.importances_std
}).sort_values(by='Importance_Mean', ascending=False)

top_10 = importances_df.head(10).copy()

# 3. Visualización Plotly
fig_imp = go.Figure()
fig_imp.add_trace(go.Bar(
    y=top_10['Feature'], x=top_10['Importance_Mean'], orientation='h',
    marker=dict(
        color=top_10['Importance_Mean'], colorscale='Viridis', showscale=True,
        colorbar=dict(title="Importance", len=0.8, x=1.02, titlefont=dict(size=12))
    ),
    error_x=dict(
        type='data', array=top_10['Importance_Std'], visible=True,
        color='#6B7B8C', thickness=1.5
    ),
    text=[f"{v:.4f}" for v in top_10['Importance_Mean']],
    textposition='outside', textfont=dict(size=11, color='#2C3E50'),
    hovertemplate="
<b>{y}</b><br>Drop in ROC-AUC: {x:.4f} ±{error_x.array:.4f}<extra></extra>"
))
fig_imp.update_layout(
    title_text=f"📊 Top 10 Variables (Permutation Importance) – {best_model_name}",
    xaxis_title='Decrease in ROC-AUC when permuting (Mayor = Más Importante)',
    yaxis=dict(autorange='reversed', tickfont=dict(size=12, color='#2C3E50')),
    height=55 * len(top_10) + 150, showlegend=False,
    margin=dict(t=80, b=60, l=150, r=100),
    plot_bgcolor='#FAFAFA', paper_bgcolor='FFFFFF'
)
fig_imp.show(config=CONFIG_PLOTLY)

# =====
# FASE 11.2: INVESTIGACIÓN DE LA SEÑAL DE ALARMA DE 'Name'
# =====
# 📌 ANÁLISIS AVANZADO DE MLOps:
# Si 'Name' aparece como muy importante, un auditor novato pensaría
# que hay Data Leakage (que el modelo memorizó nombres).
# Pero hay una explicación técnica más profunda.

name_importance = importances_df[
    importances_df['Feature'] == 'Name'
]['Importance_Mean'].values

print("\n🚨 AUDITORÍA DE LA VARIABLE 'Name':")
if len(name_importance) > 0 and name_importance[0] > 0.01:
    print(" ⚠️ ALERTA: 'Name' tiene alta importancia en Permutation Importance.")
    print(" ¿El modelo está memorizando nombres? NO. Aquí está la explicación técnica:")
    print(" 1. Permutation Importance opera sobre X_test (datos crudos).")
    print(" 2. Al permutar 'Name', el TitanicFeatureEngineer extrae un 'Title' incorrecto.")
    print(" 3. Ese 'Title' erróneo entra al clasificador y destruye la predicción.")
    print(" 4. Conclusión: 'Name' es importante porque es el CONTENDOR de 'Title'.")

    # Verificación de seguridad: Name NO debe llegar al clasificador
    fe_output_cols = list(
        TitanicFeatureEngineer().fit(X_train).transform(X_train).columns
    )
    if 'Name' in fe_output_cols:
        print(" ❌ ERROR CONFIRMADO: 'Name' NO fue eliminada por el FeatureEngineer!")
    else:
        print(" ✅ OK: 'Name' es correctamente eliminada por TitanicFeatureEngineer.")
        print(" No hay Data Leakage. La importancia es un efecto cascada del Pipeline.")
else:
    print(" ✅ 'Name' no muestra importancia significativa directa.")

# — Coeficientes de LR como referencia complementaria —
# Los coeficientes operan sobre el espacio transformado (features
# escaladas), por eso son comparables entre sí en magnitud.
lr_model = best_model_pipe.named_steps['classifier']
preprocessor = best_model_pipe.named_steps['preprocessor']

cat_feature_names = (
    preprocessor
    .named_transformers_['cat']
    .named_steps['onehot']

```



```

        .get_feature_names_out(cat_cols)
        .tolist()
    )
    all_feature_names = num_cols + cat_feature_names

    coef_df = pd.DataFrame({
        'Feature': all_feature_names,
        'Coeficiente': lr_model.coef_[0]
    })
    coef_df['|Coef|'] = coef_df['Coeficiente'].abs()
    coef_df = coef_df.sort_values('|Coef|', ascending=False).head(10).reset_index(drop=True)

    print("\n📊 Top 10 coeficientes directos de Logistic Regression")
    print("    (sobre espacio transformado – complementa la Permutation Importance):")
    display(coef_df[['Feature', 'Coeficiente', '|Coef|']].round(4))

    print("\n📖 Interpretación Histórica y de Negocio:")
    print("    • 'Sex' y 'Pclass' dominan la predicción a nivel de variable original.")
    print("    • Esto refleja fielmente la realidad histórica: el protocolo 'women and children first'")
    print("    y la proximidad física de la 1ra/2da clase a las cubiertas superiores con los botes.")
    print("    • 'Cabin' (proxy de cubierta/estatus) y 'Fare_Per_Person' también son cruciales.")
    print("=" * 70)

```

```

# =====
# FASE 12: TABLA EJECUTIVA FINAL CORREGIDA
# =====
print("=" * 70)
print("📊 FASE 12: COMPARACIÓN FINAL (EXECUTIVE SUMMARY)")
print("=" * 70)

```

```

print("🎯 Objetivo: Unificar todas las métricas (Train, CV, Test, Macro, PR-AUC)")
print("    en una sola tabla de la verdad, ordenada por ROC-AUC.")

# df_diag tiene columnas float (CV Score Mean, CV Score Std).
# El merge funciona correctamente; no hay NaN por conversión fallida.
df_executive = df_test[[
    'Modelo', 'Optimizado', 'Accuracy', 'F1_Macro', 'ROC_AUC', 'PR_AUC'
]].copy()

# Merge con diagnóstico (floats limpios)
df_executive = df_executive.merge(
    df_diag[['Modelo', 'Train Score', 'CV Score Mean', 'CV Score Std', 'GAP']],
    on='Modelo', how='left'
)
df_executive = df_executive.sort_values('ROC_AUC', ascending=False).reset_index(drop=True)

# Columna de CV legible (para display)
df_executive['CV Acc'] = df_executive.apply(
    lambda r: f"{r['CV Score Mean']:.4f} ± {r['CV Score Std']:.4f}", axis=1
)

# Preparar DataFrame para visualización
df_exec_display = df_executive[[
    'Modelo', 'Optimizado', 'Train Score', 'CV Acc',
    'Accuracy', 'GAP', 'F1_Macro', 'ROC_AUC', 'PR_AUC'
]].copy()

# Formatear columnas numéricas para display
for col in ['Train Score', 'Accuracy', 'F1_Macro', 'ROC_AUC', 'PR_AUC']:
    df_exec_display[col] = df_exec_display[col].apply(lambda x: f"{x:.4f}")
df_exec_display['GAP'] = df_exec_display['GAP'].apply(lambda x: f"{x:.1%}")

# Colores de celda (Oro para el 1º, Plata para el 2º, amarillo suave para el mejor ROC-AUC)
cell_colors_e = []
for col in df_exec_display.columns:
    col_colors = []
    for idx in range(len(df_exec_display)):
        if col == 'Modelo':
            if idx == 0: col_colors.append('#FFD700') # 🏆 Oro
            elif idx == 1: col_colors.append('#C0C0C0') # 🥈 Plata
            else: col_colors.append('FFFFFF')
        elif col == 'ROC_AUC' and idx == 0:
            col_colors.append('#FFF8DC') # Amarillo claro suave
        else:
            col_colors.append('FFFFFF')
    cell_colors_e.append(col_colors)

# Crear tabla Plotly
table_h = 40 * len(df_exec_display) + 130
fig_exec = go.Figure(data=[go.Table(
    header=dict(
        values=[f"<b>{c}</b>" for c in df_exec_display.columns],
        fill_color=MATLAB_COLORS[0], align='center',
        font=dict(color='white', size=11, family='Arial'), height=40
    ),
    cells=dict(
        values=[df_exec_display[col] for col in df_exec_display.columns],
        fill_color=cell_colors_e, align='center',
        font=dict(size=11, color='#2C3E50', family='Arial'), height=35
    )
)])
fig_exec.update_layout(
    title_text='🏆 Tabla Ejecutiva Final: Rendimiento Integral de Modelos',
    title_font=dict(size=16, color='#1A2B3C'),
    height=table_h,
    margin=dict(t=80, b=40, l=40, r=40),
    paper_bgcolor='FFFFFF'
)
fig_exec.show(config=CONFIG_PLOTLY)

# Extraer métricas del modelo ganador para el resumen
best_model = df_executive.iloc[0]['Modelo']
best_auc = float(df_executive.iloc[0]['ROC_AUC'])
best_f1 = float(df_executive.iloc[0]['F1_Macro'])
best_pr_auc = float(df_executive.iloc[0]['PR_AUC'])
worst_gap_row = df_executive.loc[df_executive['GAP'].idxmax()]

print("\n🔴 OBSERVACIONES DEL AUDITOR:")
print(f"🏆 Modelo Ganador: {best_model}")
print(f"ROC-AUC: {best_auc:.4f}")
print(f"PR-AUC: {best_pr_auc:.4f}")
print(f"F1-Macro: {best_f1:.4f}")

```

```

print(f" ⚠ Mayor riesgo de overfitting: {worst_gap_row['Modelo']} "
      f"(GAP Train-Test: {worst_gap_row['GAP']:.1%})")
print(f" 💡 La tabla está ordenada por ROC-AUC (métrica reina en datasets desbalanceados).")
print(f"     PR-AUC complementa la evaluación para la clase minoritaria.")
print("=" * 70)

```

```

# =====
# FASE 13: MLOPS AUDIT + SERIALIZACIÓN
# =====
import joblib
import os

print("=" * 70)
print(" ☐ FASE 13: CONTROL DE CALIDAD (MLOps AUDIT)")
print("=" * 70)
print(" 🎯 Objetivo: Auditar la integridad metodológica del pipeline completo")
print("     y serializar el modelo ganador para producción.")

# — 13.1 Serialización del modelo ganador —————
# 📌 Guardar el pipeline completo
# (FeatureEngineer + Preprocessor + Classifier) para producción.
MODEL_PATH = f'titanic_pipeline_{best_model_name.lower().replace(" ", "_")}.joblib'
joblib.dump(best_model_pipe, MODEL_PATH)
model_size_kb = os.path.getsize(MODEL_PATH) / 1024
print(f"\n 📦 Modelo serializado: {MODEL_PATH} ({model_size_kb:.1f} KB)")

# — 13.2 Verificar que el modelo cargado predice igual ———
loaded_pipe = joblib.load(MODEL_PATH)
y_pred_loaded = loaded_pipe.predict(X_test)
y_pred_orig = best_model_pipe.predict(X_test)
predictions_match = (y_pred_loaded == y_pred_orig).all()
print(f" ✅ Predicciones idénticas tras carga: {predictions_match}")
assert predictions_match, 'ERROR: el modelo cargado produce predicciones diferentes!'

# — 13.3 Ejemplo de inferencia en producción —————
# Simular un nuevo pasajero con los campos originales del dataset
nuevo_pasajero = pd.DataFrame([
    'PassengerId': 999,
    'Pclass': 1,
    'Name': 'Test, Mr. John Smith',
    'Sex': 'male',
    'Age': 35.0,
    'SibSp': 1,
    'Parch': 0,
    'Ticket': 'PC 17599',
    'Fare': 71.28,
    'Cabin': 'C85',
    'Embarked': 'C'
])
proba_sobrevivir = loaded_pipe.predict_proba(nuevo_pasajero)[0, 1]

```

```
prediccion      = 'Sobrevivió' if proba_sobrevivir >= 0.5 else 'Murió'
print(f"\n 🧠 Inferencia en producción (ejemplo):")
print(f"   Pasajero: {nuevo_pasajero['Name'].values[0]}")
print(f"   P(Survived): {proba_sobrevivir:.4f}")
print(f"   Predicción: {prediccion}")
```

— 13.4 Checklist MLOps Extendido —————

```
audit_checklist = pd.DataFrame({
    'Protocolo MLOps': [
        '🔒 Data Leakage',
        '🔄 Reproducibilidad',
        '⚖️ Manejo de Desbalance',
        '🔍 Validación Cruzada',
        '🧠 Interpretabilidad',
        '📊 Métricas Robustas (ROC+PR)',
        '🏗️ Arquitectura de Pipeline',
        '🏆 Comparación Justa de Modelos',
        '📈 Intervalos de Confianza',
        '⚙️ Threshold Optimization',
        '💾 Serialización del Modelo',
        '✅ Verificación Post-Carga',
    ],
})
```